Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

pede um programa em c com threads para coordenar o embarque de passageiros em um vagão de metrô, respeitando sua capacidade e aguardando a conclusão dos embarques antes da partida, eu entreguei código-fonte, o programa de testes, o Makefile, o readme e o relatório em pdf.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
- pthread_mutex_init	OK	./sync_car
- pthread_mutex_lock	OK	./sync_car
- pthread_mutex_unlock	OK	./sync_car
- pthread_cond_init	OK	./sync_car
- pthread_cond_wait	OK	./sync_car
- pthread_cond_broadcast	OK	./sync_car
- pthread_cond_signal	OK	./sync_car

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car.c → Implementa as quatro funções de sincronização do embarque**
2. **sync_car.h → Define a estrutura da estação e declara as funções.**
3. **sync_car-runner.c → Executa os testes fornecidos pelo professor.**
4. **Makefile → Compila o executável sync_car e permite sua limpeza.**
5. **README.md → Descreve os arquivos, o ambiente e os comandos de uso.**
6. **evidencias.log → Registra comandos e resultados do terminal.**

Decisões técnicas:

1. **Um mutex por estação → proteger os contadores compartilhados → evita reservas acima da capacidade**
2. **Variáveis de condição → aguardar vagas e confirmações → suspende as threads sem espera ocupada**
3. **Separar reserva e confirmação → permitir embarques simultâneos → o vagão aguarda todos os embarques autorizados antes de partir**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Mutex e variáveis de condição	Proteger os contadores, aguardar vagas e confirmar os embarques antes da partida.	

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Compilar o programa.	FALHA	Estrutura duplicada e tipos incompatíveis.	04/11	LINHAS 32–61
2	S1	Ajustar o cabeçalho e recompilar.	FALHA	Tipo station não reconhecido.	4/11	LINHAS 128–186
3	S1	Corrigir a estrutura e recompilar.	FALHA	Nome da função diferente do esperado pelo runner.	4/11	LINHAS 305–308
4	S1	Compilar e executar após as correções.	OK		4/11	LINHAS 375–385

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

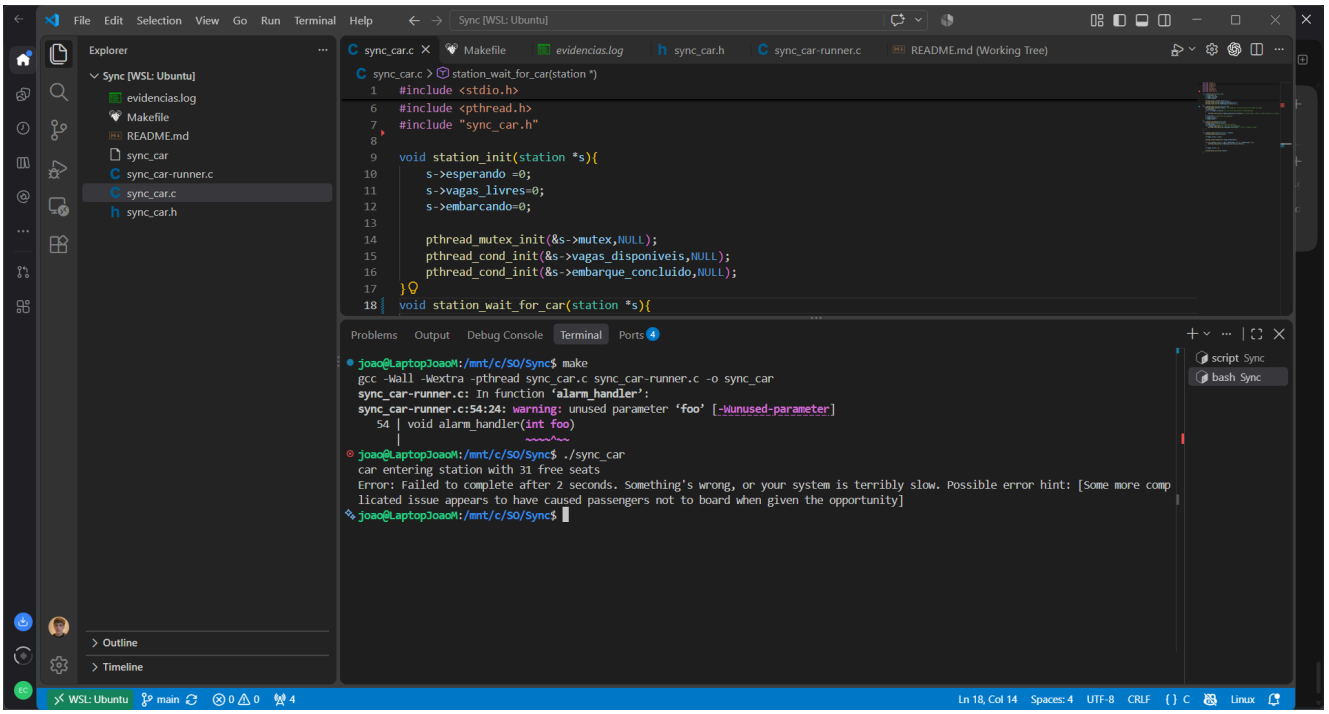
6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints



Eu me esqueci de colocar o mutex_unlock em 2 funcoes

```
joao@LaptopJoaoM:/mnt/c/SO/Sync$ ./sync_car
car entering station with 44 free seats
car departed station with 44 new passenger(s) (expected 44)
car entering station with 25 free seats
car departed station with 25 new passenger(s) (expected 25)
car entering station with 24 free seats
car departed station with 24 new passenger(s) (expected 24)
car entering station with 40 free seats
car departed station with 7 new passenger(s) (expected 7)
Looks good!
joao@LaptopJoaoM:/mnt/c/SO/Sync$
```

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Usei IA como ferramenta de apoio para compreender mutex e variáveis de condição, discutir a organização das funções e receber sugestões de código e correções.

“Oriente a estrutura e as funções do programa”

“Explique mutex e variáveis de condição.”

“Explique o funcionamento das funções de sincronização. “

Compilei e executei o programa conferi a quantidade de passageiros embarcados em cada vagão e a mensagem final Looks good!

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão: Com mais tempo, criaria testes adicionais para explorar diferentes ordens de execução das threads, a principal decisão técnica foi usar um mutex por estação para proteger os contadores compartilhados, as variáveis de condição permitiram aguardar vagas e confirmações sem espera ocupada. Separar a reserva da confirmação foi necessário para permitir vários embarques em andamento. Na próxima atividade, conferiria desde o início os nomes e os tipos das funções no cabeçalho e também verificaria se cada caminho das funções libera o mutex, evitando o travamento que ocorreu.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Adicionaria testes para estação vazia, zero vagas e passageiros excedendo a capacidade. iria verificar se várias reservas podem ocorrer antes da primeira confirmação de embarque. executaria o runner 100 vezes e registraria quantas terminaram com Looks good!. Trataria o aviso sobre foo e verificaria os retornos das funções de inicialização. Registraria data, comando e resultado individuais de cada teste no arquivo de evidências.log

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.